

1. Intro (What we're building)
 2. Prerequisite topics
 3. Project demo (output preview)
 4. Code explanation **line by line**
 5. Common errors & tips
 6. Final summary
-

YOUTUBE SCRIPT (LINE-BY-LINE EXPLANATION)

◆ INTRO (0:00 – 0:30)

Script (Speak this):

“In this video, we are going to build a real-world Python automation project where we read data from an Excel file, filter records, convert them into an HTML table, and automatically send an email with that Excel file as an attachment using Python. This type of automation is widely used in companies for attendance reports, alerts, and daily monitoring.”

◆ PREREQUISITE TOPICS (0:30 – 1:30)

Script:

“Before starting, make sure you understand these basic topics:”

Prerequisites

- Python basics (functions, variables, if-else)
- What is SMTP (Simple Mail Transfer Protocol)
- Basic Pandas (reading Excel files)
- Environment variables & .env file
- Gmail App Password (important!)

“Don't worry — even if you are a beginner, I'll explain everything step by step.”

◆ LIBRARIES USED (1:30 – 2:00)

```
import smtplib
import pandas as pd
import os
from email.message import EmailMessage
from pathlib import Path
from dotenv import load_dotenv
```

Script:

“Let me explain each library one by one.”

- smtplib → Used to connect to Gmail SMTP server
 - pandas → Used to read Excel files
 - os → Used to read environment variables
 - EmailMessage → Helps create professional emails
 - Path → Used to safely handle file names
 - dotenv → Loads email credentials securely
-

◆ **LOADING EMAIL CREDENTIALS (2:00 – 3:00)**

```
load_dotenv()
```

```
SENDER_EMAIL = os.getenv("SENDER_EMAIL")
```

```
SENDER_PASSWORD = os.getenv("SENDER_PASSWORD")
```

Script:

“Instead of writing email and password directly in the code, we store them securely inside a .env file.

This is very important for security and industry best practices.”

```
if not SENDER_EMAIL or not SENDER_PASSWORD:
```

```
    raise ValueError("Email credentials missing in .env file")
```

“If credentials are missing, the program will stop immediately to avoid runtime errors.”

◆ SMTP CONFIGURATION (3:00 – 3:30)

```
SMTP_SERVER = "smtp.gmail.com"
```

```
SMTP_PORT = 587
```

Script:

“Here we configure Gmail’s SMTP server.
Port 587 is used for secure TLS communication.”

◆ EXCEL CONFIGURATION (3:30 – 4:00)

```
EXCEL_FILE = "data.xlsx"
```

```
SHEET_NAME = "Attendance"
```

```
MIN_WORK_HOURS = 8
```

Script:

“This file contains employee attendance data.
We define minimum work hours as 8 — anyone below this will trigger an alert.”

◆ READING & FILTERING EXCEL DATA (4:00 – 5:30)

```
def read_excel_data():
```

“This function reads Excel data and filters employees who worked less hours.”

```
df = pd.read_excel(EXCEL_FILE, sheet_name=SHEET_NAME)
```

“We load the Excel sheet using Pandas.”

```
short_hours_df = df[df["Total Hours"] < MIN_WORK_HOURS]
```

“Here we apply filtering logic.

Only employees working less than 8 hours are selected.”

```
return short_hours_df
```

◆ CONVERT DATAFRAME TO HTML (5:30 – 7:00)

```
def dataframe_to_html(df):
```

“Now we convert Excel data into a beautiful HTML table.”

```
def highlight_hours(value):
```

“This function highlights hours in red if they are below the limit.”

```
styled_df = df.style.map(highlight_hours, subset=["Total Hours"])
```

“Pandas styling helps us visually highlight critical data.”

```
return styled_df.to_html(index=False)
```

◆ EMAIL SENDING FUNCTION (7:00 – 10:00)

```
def send_email(to, cc, subject, html_body, attachment):
```

“This function handles email creation, HTML content, and attachments.”

```
msg = EmailMessage()
```

“Creates a new email object.”

```
msg["From"] = SENDER_EMAIL
```

```
msg["To"] = ", ".join(to)
```

```
msg["Cc"] = ", ".join(cc)
```

```
msg["Subject"] = subject
```

“These are standard email headers.”

```
msg.set_content("Please view this email in HTML format.")
```

```
msg.add_alternative(html_body, subtype="html")
```

“This ensures email works even if HTML is not supported.”

◆ ATTACHING EXCEL FILE (10:00 – 11:00)

```
with open(attachment, "rb") as file:
```

“We open the Excel file in binary mode.”

```
msg.add_attachment(
```

```
    file.read(),
```

```
    maintype="application",
```

```
    subtype="octet-stream",
```

```
    filename=Path(attachment).name
```

```
)
```

“This attaches the Excel file to the email.”

◆ SMTP CONNECTION & SENDING EMAIL (11:00 – 12:00)

with smtplib.SMTP(SMTP_SERVER, SMTP_PORT) as server:

“We establish connection with Gmail server.”

```
server.starttls()
```

```
server.login(SENDER_EMAIL, SENDER_PASSWORD)
```

```
server.send_message(msg, to_addrs=recipients)
```

“TLS encryption, login, and finally send the email.”

◆ MAIN EXECUTION BLOCK (12:00 – 13:30)

```
if __name__ == "__main__":
```

“This ensures code runs only when executed directly.”

```
short_hours_df = read_excel_data()
```

“Read and filter Excel data.”

```
if short_hours_df.empty:
```

“If no one has short hours, we exit.”

```
html_table = dataframe_to_html(short_hours_df)
```

“Convert data into HTML table.”

```
send_email(...)
```

“Finally send the email with attachment.”

⚠ COMMON ERRORS & TIPS (13:30 – 14:30)

- Use **Gmail App Password**, not your real password
 - Excel column names must match exactly
 - Enable Less Secure App alternatives via App Password
 - Always test with your own email first
-

FINAL SUMMARY (14:30 – 15:00)

Script:

“In this video, we automated Excel reporting, HTML email formatting, and file attachments using Python.

This project is extremely useful for real-world corporate automation.”

CTA (Ending)

“If this video helped you, like, share, and subscribe.

Comment ‘AUTOMATION’ if you want the GitHub code and more such projects.”